

Binary Tree Documentation

Program Design Discussion

This program uses a modular, procedural design with pointer-based binary trees. The user enters a preorder string representation of a binary tree, where each character represents a node and each period "." represents an empty subtree. The program validates the input, builds the tree, and displays preorder, inorder, postorder, and level-order traversals. The main control flow is handled by the runProgram function. The main function stays simple and only calls runProgram. Helper functions are used for input validation, tree construction, traversals, and tree property checks.

Data Structure Design Discussion

Each binary tree node is represented using the TreeNode structure. Each node contains char data, TreeNode* left, and TreeNode* right. The program also uses typedef TreeNode* BinaryTree so the root of a tree can be referred to as BinaryTree. Nodes are created dynamically using new and deleted with clearTree to prevent memory leaks. The program also uses a linked queue for level-order traversal and for checking whether the tree is complete. The queue stores pointers to tree nodes and supports enqueue, dequeue, empty check, and clear operations.

Tree Building Structure

The tree is built by the BuildTree function using a recursive helper function. The input string is cleaned first by trimming and removing spaces. The recursive function reads the preorder string from left to right. If the current character is ".", the function returns an empty subtree. Otherwise, it creates a node, builds the left subtree, and then builds the right subtree. An index variable keeps track of the current position in the string. After building, the program checks that the entire string was used so extra characters are not accepted.

Input Validation Structure

Input validation is used for both menu choices and tree strings. The readIntegerLine function makes sure the user enters a valid menu option within the allowed range. The tree string is invalid if it is empty, ends too early, contains invalid node characters, or has extra characters after the tree is complete. A period is only used to represent an empty subtree and is never stored as a node.

Tree Operation Structure

The program includes functions to analyze the binary tree. IsBST determines whether the tree is a binary search tree without using inorder traversal or sorting. It checks each node using allowed minimum and maximum values. node_count returns the total number of nodes in the tree. height returns the height of the tree. An empty tree has height 0. IsFullBT returns true if every node has either 0 children or 2 children. IsCompleteBT returns true if the tree is complete. It uses level-order traversal and checks that no node appears after a missing child in a way that violates completeness.

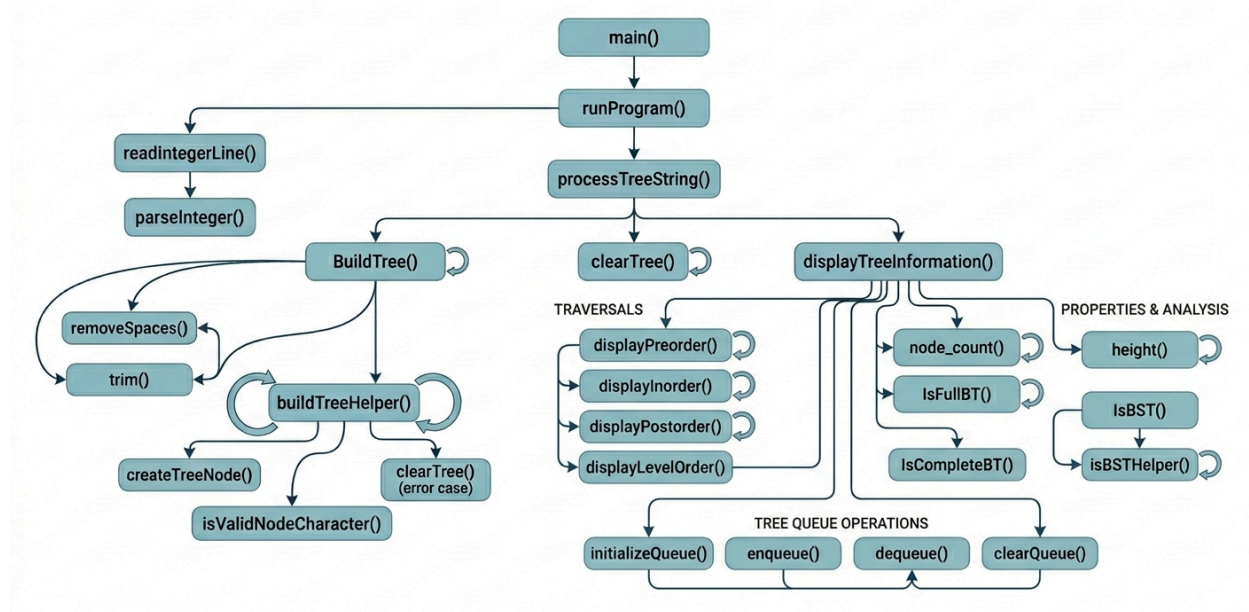
Traversal and Output Structure

The program has separate functions for each traversal. displayPreorder prints the tree in root, left, right order. displayInorder prints the tree in left, root, right order. displayPostorder prints the tree in left, right, root order. displayLevelOrder prints the tree level by level using the linked queue. The displayTreeInformation function organizes the final output by displaying all traversals and tree properties in one place.

Program Control Structure

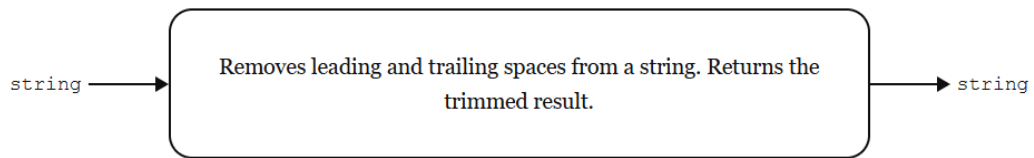
The runProgram function controls the overall program. It displays the menu, reads the user's choice, accepts a preorder tree string, processes the tree, displays the results, and clears memory before returning to the menu. The basic flow is: main calls runProgram, runProgram displays the menu, the user enters a preorder tree string, BuildTree creates the binary tree, the program displays traversals and tree information, the tree is deleted from memory, and the menu repeats until the user chooses to quit.

Hierarchy Chart:

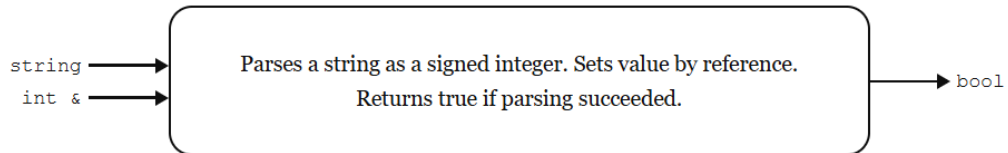


Procedural Specification:

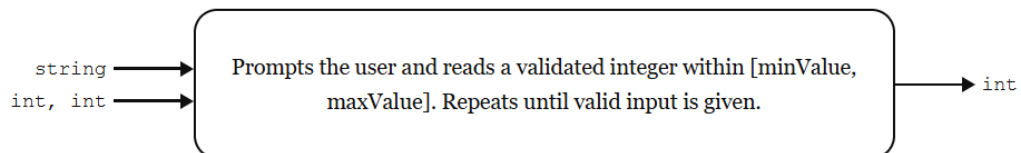
trim



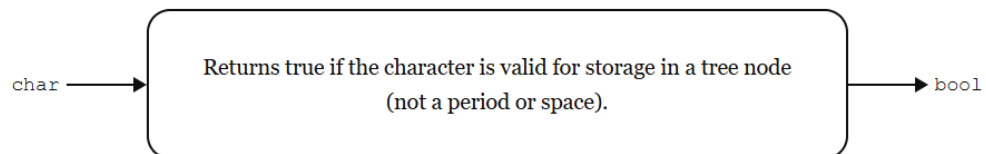
parseInteger



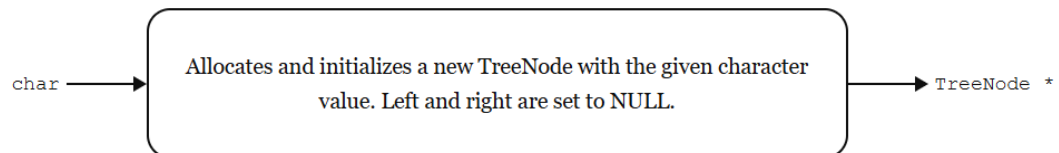
readIntegerLine



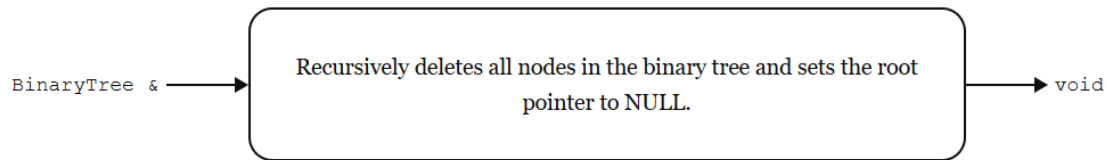
isValidNodeCharacter



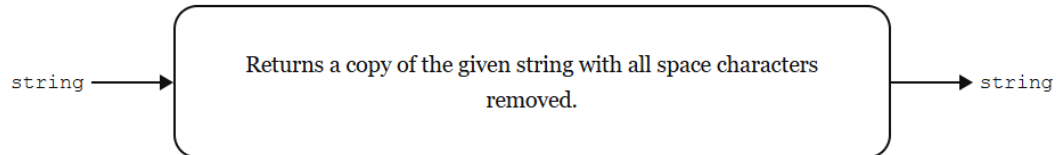
createTreeNode



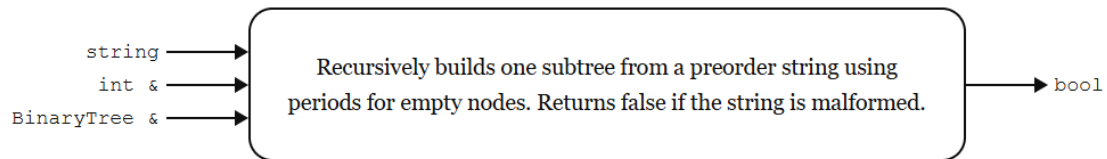
clearTree



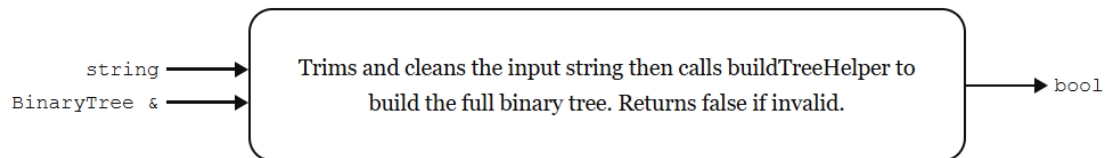
removeSpaces



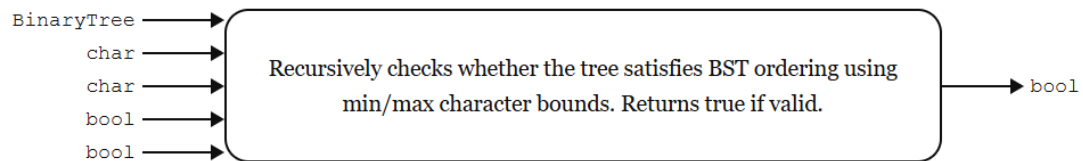
buildTreeHelper



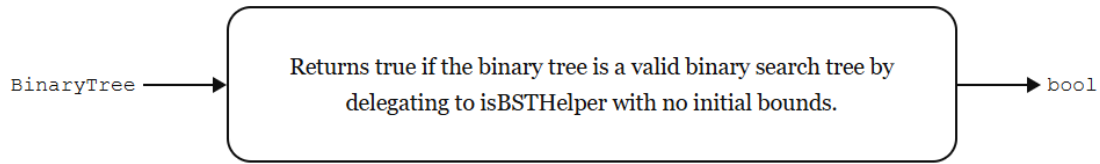
BuildTree



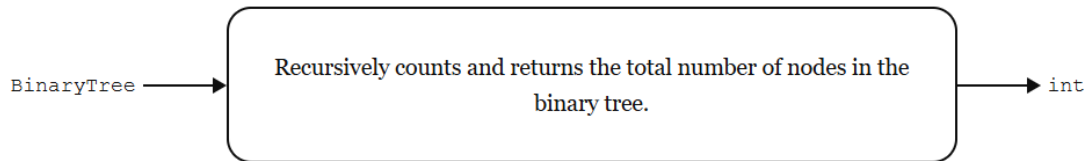
isBSTHelper



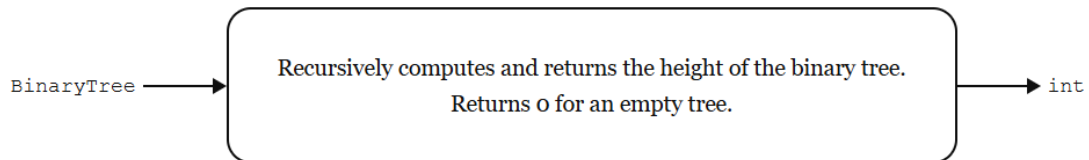
IsBST



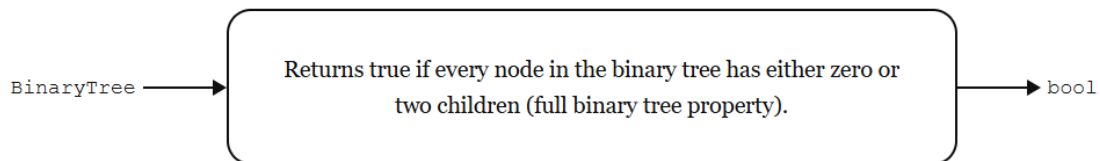
node_count



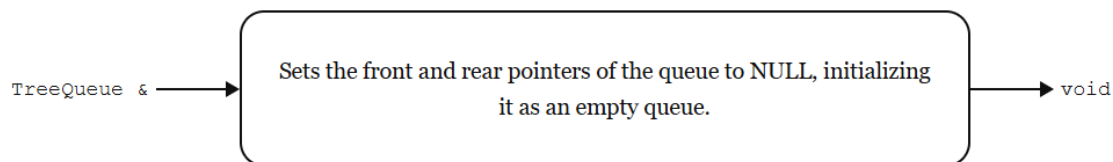
height



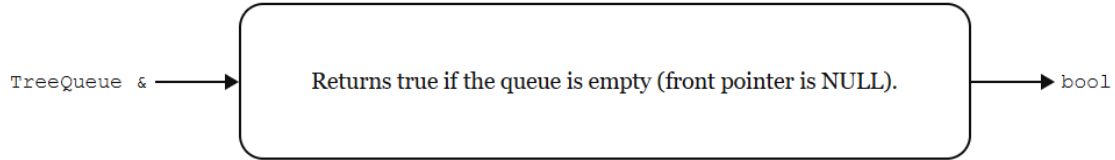
IsFullBT



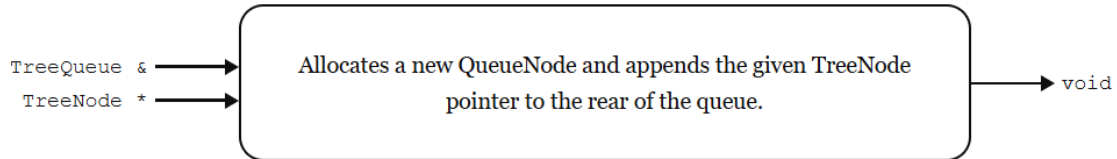
initializeQueue



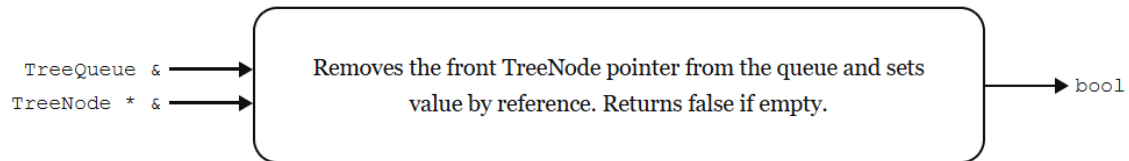
isEmptyQueue



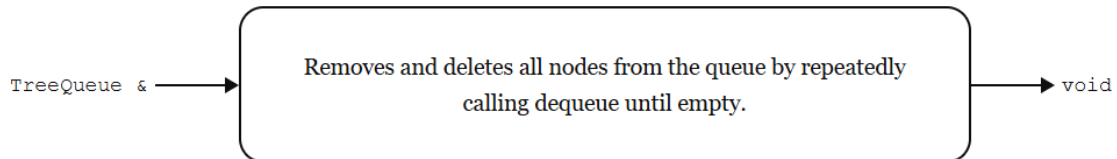
enqueue



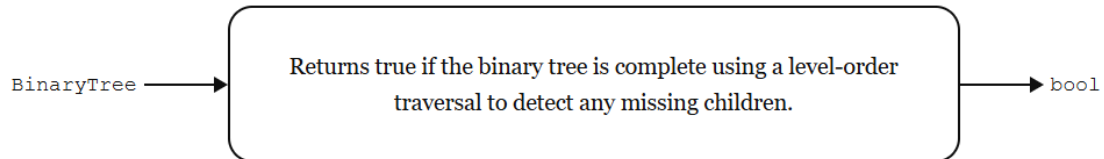
dequeue



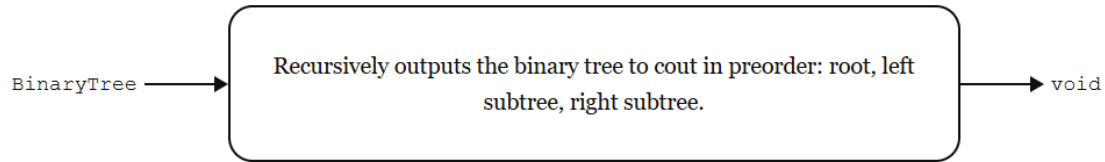
clearQueue



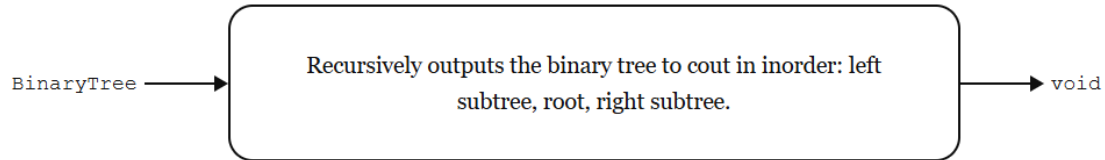
IsCompleteBT



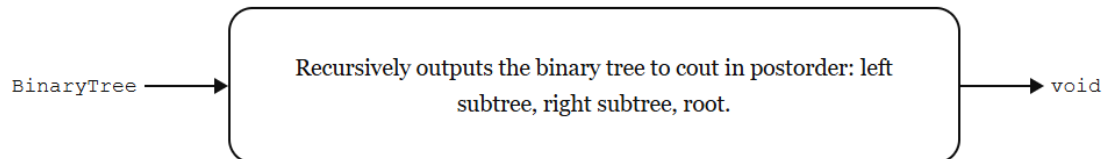
displayPreorder



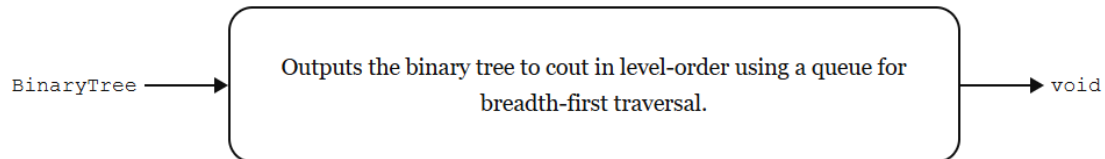
displayInorder



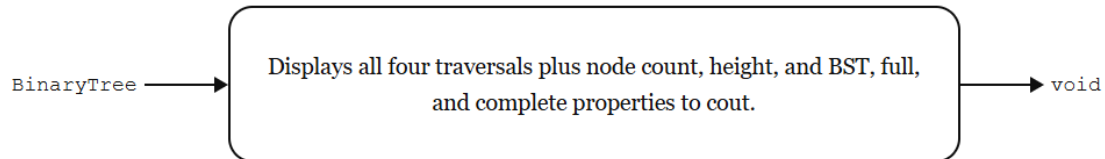
displayPostorder

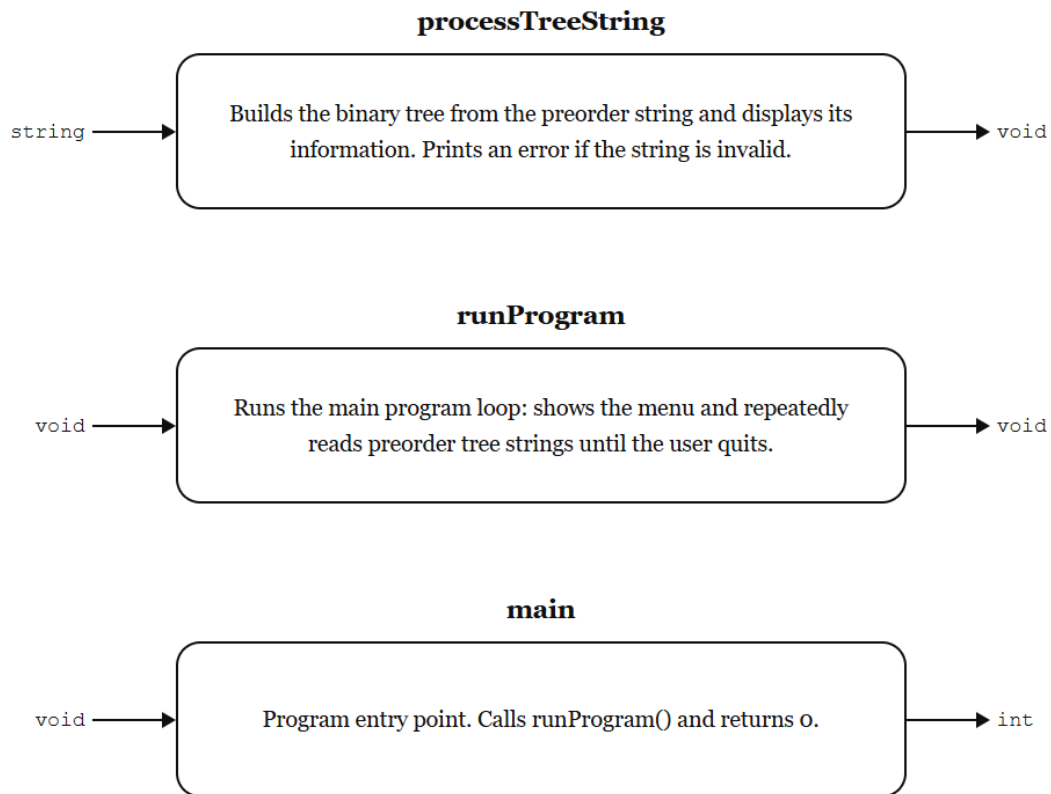


displayLevelOrder



displayTreeInformation





Test Cases with Expected Outputs

Test Case 1 — Exit Immediately

Inputs: main menu choice = 0

Justification: Confirms the program exits correctly when the user chooses to quit.

Expected Output: Program exits cleanly without requesting a tree string.

Test Case 2 — Empty Tree

Inputs: main menu choice = 1; preorder string = .

Justification: Confirms the program correctly accepts a period as the representation of an empty binary tree.

Expected Output: Program displays an empty tree with node count = 0, height = 0, BST = true, full binary tree = true, and complete binary tree = true.

Test Case 3 — Single Node Tree

Inputs: main menu choice = 1; preorder string = A..

Justification: Confirms the program correctly builds the simplest non-empty binary tree with one root node.

Expected Output: Preorder = A, Inorder = A, Postorder = A, Level-order = A, node count = 1, height = 1, BST = true, full binary tree = true, and complete binary tree = true.

Test Case 4 — Root with Only Left Child

Inputs: main menu choice = 1; preorder string = AB...

Justification: Confirms the program correctly builds a tree where the root has a left child but no right child.

Expected Output: Preorder = AB, Inorder = BA, Postorder = BA, Level-order = AB, node count = 2, height = 2, BST = true, full binary tree = false, and complete binary tree = true.

Test Case 5 — Root with Only Right Child

Inputs: main menu choice = 1; preorder string = A.B..

Justification: Confirms the program correctly builds a tree where the root has no left child but has a right child.

Expected Output: Preorder = AB, Inorder = AB, Postorder = BA, Level-order = AB, node count = 2, height = 2, BST = true, full binary tree = false, and complete binary tree = false.

Test Case 6 — Full Three-Node Tree

Inputs: main menu choice = 1; preorder string = AB..C..

Justification: Confirms the program correctly builds a full binary tree with one root and two children.

Expected Output: Preorder = ABC, Inorder = BAC, Postorder = BCA, Level-order = ABC, node count = 3, height = 2, BST = true, full binary tree = true, and complete binary tree = true.

Test Case 7 — Complete Tree with Three Levels

Inputs: main menu choice = 1; preorder string = ABD..E..C..

Justification: Confirms the program correctly identifies a complete binary tree that is not full because one internal node has no children while another has two.

Expected Output: Preorder = ABDEC, Inorder = DBEAC, Postorder = DEBCA, Level-order = ABCDE, node count = 5, height = 3, BST = false, full binary tree = false, and complete binary tree = true.

Test Case 8 — Perfect Binary Tree

Inputs: main menu choice = 1; preorder string = ABD..E..CF..G..

Justification: Confirms the program correctly builds and analyzes a perfect binary tree.

Expected Output: Preorder = ABDECFG, Inorder = DBEAF CG, Postorder = DEBFGCA, Level-order = ABCDEFG, node count = 7, height = 3, BST = false, full binary tree = true, and complete binary tree = true.

Test Case 9 — Valid BST with Multiple Levels

Inputs: main menu choice = 1; preorder string = DB..FA..G..

Justification: Confirms the program correctly identifies a valid binary search tree with both left and right subtrees.

Expected Output: Preorder = DBFAG, Inorder = BDAGF, Postorder = BAGFD, Level-order = DBFAG, node count = 5, height = 3, BST = false, full binary tree = false, and complete binary tree = false.

Test Case 10 — Valid Larger BST

Inputs: main menu choice = 1; preorder string = DB..F.E..G..

Justification: Confirms the program correctly identifies a larger valid binary search tree.

Expected Output: Preorder = DBFEG, Inorder = DBEFG, Postorder = BEGFD, Level-order = DBFEG, node count = 5, height = 3, BST = true, full binary tree = false, and complete binary tree = false.

Test Case 11 — Not a BST Because Left Child Is Too Large

Inputs: main menu choice = 1; preorder string = DA..B..

Justification: Confirms the program rejects a tree as a BST when the right subtree contains a value smaller than the root.

Expected Output: Preorder = DAB, Inorder = ADB, Postorder = ABD, Level-order = DAB, node count = 3, height = 2, BST = false, full binary tree = true, and complete binary tree = true.

Test Case 12 — Not a BST Because Deep Left Subtree Violates Root Rule

Inputs: main menu choice = 1; preorder string = DB..CA...

Justification: Confirms the BST check validates the entire subtree, not just direct parent-child relationships.

Expected Output: Preorder = DBCA, Inorder = BDAC, Postorder = BADC, Level-order = DBCA, node count = 4, height = 3, BST = false, full binary tree = false, and complete binary tree = false.

Test Case 13 — Left-Skewed Tree

Inputs: main menu choice = 1; preorder string = ABCD.....

Justification: Confirms the program correctly handles a tree where every node only has a left child.

Expected Output: Preorder = ABCD, Inorder = DCBA, Postorder = DCBA, Level-order = ABCD, node count = 4, height = 4, BST = true, full binary tree = false, and complete binary tree = false.

Test Case 14 — Right-Skewed Tree

Inputs: main menu choice = 1; preorder string = A.B.C.D..

Justification: Confirms the program correctly handles a tree where every node only has a right child.

Expected Output: Preorder = ABCD, Inorder = ABCD, Postorder = DCBA, Level-order = ABCD, node count = 4, height = 4, BST = true, full binary tree = false, and complete binary tree = false.

Test Case 15 — Valid Input with Spaces

Inputs: main menu choice = 1; preorder string = A B . . C . .

Justification: Confirms the program removes spaces before building the tree.

Expected Output: Same result as AB..C.: Preorder = ABC, Inorder = BAC, Postorder = BCA, Level-order = ABC, node count = 3, height = 2, BST = true, full binary tree = true, and complete binary tree = true.

Test Case 16 — Empty Input String

Inputs: main menu choice = 1; preorder string = empty line

Justification: Confirms the program rejects blank tree input.

Expected Output: Program displays an invalid tree representation message and does not display tree traversals.

Test Case 17 — Input Ends Too Early

Inputs: main menu choice = 1; preorder string = AB..

Justification: Confirms the program rejects a preorder representation that does not include all required empty subtree markers.

Expected Output: Program displays an invalid tree representation message and does not display tree traversals.

Test Case 18 — Extra Characters After Complete Tree

Inputs: main menu choice = 1; preorder string = A..B

Justification: Confirms the program rejects input that contains extra characters after a complete tree has already been built.

Expected Output: Program displays an invalid tree representation message and does not display tree traversals.

Test Case 19 — Invalid Node Character

Inputs: main menu choice = 1; preorder string = A#..

Justification: Confirms the program rejects unsupported characters that cannot be stored as tree nodes.

Expected Output: Program displays an invalid tree representation message and does not display tree traversals.

Test Case 20 — Invalid Menu Choice

Inputs: main menu choice = 5, then 0

Justification: Confirms the menu input validation rejects values outside the allowed range.

Expected Output: Program displays an invalid input message, reprompts for a valid menu choice, and exits when 0 is entered.

Testing Results:

```
Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 0
PS C:\Users\Brent\OneDrive\Desktop\Data Structures & Algorithms\BuildingBinaryTree>
```

```
Enter choice -> 1

Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> .

Preorder    =
Inorder     =
Postorder   =
Level-order =

Node count  = 0
Height      = 0
Is BST      = TRUE
Is Full     = TRUE
Is Complete = TRUE
```

```
Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 1
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> A..
```

```
Preorder      = A
Inorder       = A
Postorder     = A
Level-order   = A

Node count    = 1
Height        = 1
Is BST        = TRUE
Is Full       = TRUE
Is Complete   = TRUE
```

```
Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 1
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> AB..
```

```
Invalid tree representation.
```

```
Enter choice -> 1

Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> A.B..
```

```
Preorder      = AB
Inorder       = AB
Postorder     = BA
Level-order   = AB

Node count    = 2
Height        = 2
Is BST        = TRUE
Is Full       = FALSE
Is Complete   = FALSE
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> AB..C..

Preorder    = ABC
Inorder     = BAC
Postorder   = BCA
Level-order = ABC

Node count  = 3
Height      = 2
Is BST      = FALSE
Is Full     = TRUE
Is Complete = TRUE

Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 1

Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> ABD..E..C..

Preorder    = ABDEC
Inorder     = DBEAC
Postorder   = DEBCA
Level-order = ABCDE

Node count  = 5
Height      = 3
Is BST      = FALSE
Is Full     = TRUE
Is Complete = TRUE
```

```
Enter choice -> 1

Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> ABD..E..CF..G..

Preorder    = ABDECFG
Inorder     = DBEAFCG
Postorder   = DEBFGCA
Level-order = ABCDEFG

Node count  = 7
Height      = 3
Is BST      = FALSE
Is Full     = TRUE
Is Complete = TRUE
```

```
Use a period for an empty subtree.  
Example: AB..C..  
Input preorder tree string -> DB..FA..G..
```

```
Preorder    = DBFAG  
Inorder     = BDAFG  
Postorder   = BAGFD  
Level-order = DBFAG
```

```
Node count  = 5  
Height      = 3  
Is BST      = FALSE  
Is Full     = TRUE  
Is Complete = FALSE
```

Main Menu

```
0. Quit  
1. Build a binary tree from preorder input  
Enter choice ->  
Invalid input. Please try again.
```

```
Enter choice -> 1
```

```
Use a period for an empty subtree.  
Example: AB..C..  
Input preorder tree string -> DB..F.E..G..  
  
Invalid tree representation.
```

```
Use a period for an empty subtree.  
Example: AB..C..  
Input preorder tree string -> DA..B..
```

```
Preorder    = DAB  
Inorder     = ADB  
Postorder   = ABD  
Level-order = DAB
```

```
Node count  = 3  
Height      = 2  
Is BST      = FALSE  
Is Full     = TRUE  
Is Complete = TRUE
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> DB..CA...

Preorder    = DBCA
Inorder     = BDAC
Postorder   = BACD
Level-order = DBCA

Node count  = 4
Height      = 3
Is BST      = FALSE
Is Full     = FALSE
Is Complete = FALSE
```

```
Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 1
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> ABCD.....
```

```
Preorder    = ABCD
Inorder     = DCBA
Postorder   = DCBA
Level-order = ABCD

Node count  = 4
Height      = 4
Is BST      = FALSE
Is Full     = FALSE
Is Complete = FALSE
```

```
Main Menu
0. Quit
1. Build a binary tree from preorder input
Enter choice -> 1
```

```
Use a period for an empty subtree.
Example: AB..C..
Input preorder tree string -> A.B.C.D..
```

```
Preorder    = ABCD
Inorder     = ABCD
Postorder   = DCBA
Level-order = ABCD

Node count  = 4
Height      = 4
Is BST      = TRUE
Is Full     = FALSE
Is Complete = FALSE
```


Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice -> 1

Use a period for an empty subtree.

Example: AB..C..

Input preorder tree string -> A B . . C . .

Preorder = ABC

Inorder = BAC

Postorder = BCA

Level-order = ABC

Node count = 3

Height = 2

Is BST = FALSE

Is Full = TRUE

Is Complete = TRUE

Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice -> 1

Use a period for an empty subtree.

Example: AB..C..

Input preorder tree string ->

Invalid tree representation.

Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice -> 1

Use a period for an empty subtree.

Example: AB..C..

Input preorder tree string -> AB..

Invalid tree representation.

Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice -> 1

Use a period for an empty subtree.

Example: AB..C..

Input preorder tree string -> A..B

Invalid tree representation.

Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice ->

Invalid input. Please try again.

Enter choice -> A#..

Invalid input. Please try again.

Enter choice -> 5

Invalid input. Please try again.

Enter choice -> 1

Use a period for an empty subtree.

Example: AB..C..

Input preorder tree string -> A#..

Invalid tree representation.

Main Menu

0. Quit

1. Build a binary tree from preorder input

Enter choice -> 5

Invalid input. Please try again.

Enter choice -> █